TrustShield — Final Simplification Report & Execution Plan

17 Oct 2026 · Basis: `trustshield_full_handoff.zip` *(static review + trust-engine ka live run) + Antigravity AI ki report, dono ko code se cross-verify kiya gaya*

1. Verdict ek nazar mein

Tumhara core system achha hai: temporally-safe XGBoost scorer, graph features, isotonic calibration, conformal sets, TreeSHAP, FastAPI, aur 20-seed audited results. Overengineering is core ke **aas-paas** hai: ek aisi evaluation "security" layer jo sirf tests use karte hain, ek GNN jo tumhare apne benchmark mein haarta hai (phir bhi serving mein hai), 4 infra services jo default mein off hain, aur agent-generated paperwork.

Antigravity ki report **~80% sahi** hai (direction aur zyaadatar removals). Lekin 4 jagah galat ya adhoori hai, aur 6 bade issues miss karti hai (neeche). Dono ko milakar ye final plan banaya hai.

Measured impact (sirf line counts, jo maine dekhe): archive 14.8k + evaluation gate aur uske tests 2.9k + GNN files ~1.1k + orphan modules ~0.7k + infra services ~0.9k + trust-graph mock page 0.8k = **~21k lines (47.3k mein se ~45%)** bina real functionality khoye. Docker/dependency size ke numbers abhi *estimates* hain; `docker images` se before/after naapna.

2. Antigravity report: kya sahi, kya galat

Antigravity ka claim	Verdict	Verification note
GNN stack (gnn_model, hetero_gnn, temporal_gnn, phase5_hybrid) hatao	**Sahi**	Apna README bhi -0.0263 ROC-AUC dikhata hai.

Antigravity ka claim	Verdict	Verification note
"Phase 5 zero production usage"	**Galat (overstated)**	`scoring.py` mein Phase 5 `TRUSTSHIELD_SERVING_MODEL`, request body ya header se select ho sakta hai. 5 test files mein `phase5` referenced hai, to hatane par tests edit karne honge.
Neo4j / Redis default off, fallback hamesha fire hota hai	**Sahi**	`NEO4J_ENABLED` aur `REDIS_ENABLED` dono default `false`.
Prometheus + Grafana hatao	**Direction sahi, effort kam aanka**	`metrics_service` ~19 jagah wired hai (routers, features, main). `/metrics` 2 test files mein tested hai. "Latency badhata hai" wala argument kamzor hai.
sqlalchemy + psycopg2 hatao	**Sahi, aur strong**	`psycopg2` kahin import hi nahi hota; `sqlalchemy` sirf `export_dataset.py` mein.
cryptography + evaluation_gate hatao	**Sahi**	Gate sirf `test_stage354_*` import karta hai. Uske saath `test_stage35_temporal_holdout_readiness` aur `docker/Dockerfile.evaluator` bhi jaane chahiye.
sse-starlette + stream_service hatao	**Sahi + bonus finding**	`sse_starlette` repo mein kahin import nahi hota (pure dead dependency). `stream_service` fixed templates par real scoring chalata hai, yaani simulated feed.
`seed_graph.cypher` (82 KB), `seed_redis.txt` (713 KB) hatao	**Irrelevant**	Ye zip mein hain hi nahi; `.gitignore` mein generated files hain. Repo se kuch hatana nahi.
"torch hatao, transformers ki jagah `clip` package lo"	**Adhoora / galat**	CLIP ko encode karne ke liye torch chahiye (transformers ya openai-clip dono). Lekin *serving* mein CLIP scoring cached embeddings par pure numpy hai (`np.sum(u*v)`). Sahi fix: torch + transformers + Pillow ko **offline-only** requirements mein daalo (sirf `build_clip_embeddings.py` ke liye).

Antigravity ka claim	Verdict	Verification note
"~2.5 GB" / "~1.8 GB" / "60% image reduction"	**Unverified aur apne aap mein inconsistent**	Measure karke hi likho.
FAISS wrapper hatao	**Sahi**	`faiss` sirf `multimodal_clip_faiss.py` mein; Dockerfile `faiss-cpu` do baar install karta hai.
"RAG" aur "Agent" ke naam oversell hain	**Sahi**	`investigation_rag.py` keyword-matched hardcoded corpus hai; `investigation_agent.py` mein koi LLM call nahi.
Conformal / trust engine "don't touch"	**Galat**	Neeche section 3 dekho.
Frontend ke 11 pages keep	**Partly galat**	`trust-graph/page.tsx` (772 lines) hardcoded fake clusters hai.
`archive/` hatao	**Sahi**	Git history + tag kaafi hai.

3. Jo Antigravity ne miss kiya (mere findings)

1. **CI abhi tooti hui hai (verified).** `ci.yml` `trustshield_project/test_repair_pipeline_regression.py` chalata hai, jo ab `archive/historical_tests/` mein hai.
2. **Trust engine ka hand-weighted blend (verified, chalakar).** Final risk = calibrated probability aur hand-written heuristics ka weighted average. Maine engine chalaya:
   - Fraud, model 0.95 sure, device sharing nahi → final risk **0.475 → REVIEW**
   - Fraud, model 0.80 sure, sharing nahi → **0.400 → REVIEW**
   - Legit family address share kare, model 0.10 → **0.303 → REVIEW**
   - Tumhara apna result kehta hai graph lift sirf +0.011 hai aur plain counts barabar hain, phir bhi heuristics decision par haavi hain. Ye calibration (ECE ~0.02) ko decision tak pahunchne nahi deta. Ye complexity se zyada **correctness** issue hai.
3. **"neo4j" ka galat label (verified).** `scoring.py` response mein `graph_topology: "neo4j"` likhta hai jab Neo4j available ho, jabki graph features `graph_snapshot.joblib` se aate hain.

4. **Serving simulator import karti hai (verified).** `model_loader.py` startup par `product_listing_generator.generate_product_catalog()` chalata hai. Catalog ko file se load karo.
5. **Packaging hacks (verified).** `sys.path.insert`, teen jagah `try/except ImportError` import chains, aur `scoring._get_engine()` jo test patches ke liye `sys.modules["backend.main"]` kuredta hai. `_resolve_graph_features` har request par 2–4 baar chalta hai (phase5 path mein feature row do baar banta hai).
6. **Repo clutter:** `.agents/` (31 files), `skills-lock.json`, `frontend/AGENTS.md`, `audit/` (10 agent-generated reports jo purani code state ke test counts batati hain), `docs/BUG_INVENTORY.json`, `FRONTEND_IMPLEMENTATION_REPORT.md`.
7. **Models folder:** `models/v2_1/` ka koi live consumer nahi mila; `models/stage34/` sirf conformal fallback ke liye hai jabki `phase3_val_*.npy` already maujood hain; `experiments/phase41_causal` sirf ek verification script padhti hai.
8. **Scripts (23 files, ~7.8k lines) aur do synthetic-data pipelines (verified).** `train_and_save_models.py` (jo serving models banata hai) purane generators (`build_base_entities`, `inject_all_fraud`) use karta hai, jabki serving ki point-in-time history `data/synthetic_v2_1/orders.csv` padhti hai. Yaani models aur live PIT features alag datasets par ho sakte hain: **train/serve mismatch ka risk, verify karo.** 20-seed runners ka data source maine confirm nahi kiya. Baaki scripts docstrings aur imports se judge kiye hain; delete karne se pehle bodies padho.
9. `pytest` aur `httpx` production `requirements.txt` mein hain (dev-only hone chahiye); `test_phase2_suite.py` mostly un orphan modules ko zinda rakhne ke liye hai jo hat rahe hain.
10. **`.env.example` Redis/Neo4j ON karta hai (verified).** Code default `false` hai, lekin `.env.example` mein dono `true` hain aur README quickstart `cp .env.example .env` karta hai. "Default off" sirf bina `.env` ke sach hai.
11. **Demo fixtures production code mein.** `investigation_service.py` mein hardcoded `PRESET_ENTITIES` (demo orders) hain. Dossier stack (~770 lines) ka UI sirf `investigations` page hai.
12. **Conformal guarantee overclaim.** Response mein `conformal_coverage_guarantee=0.95` hardcoded constant hai, jabki calibration validation period par hui aur test out-of-time hai (exchangeability tootti hai). Ise "nominal coverage" likho.

4. Final decisions

REMOVE

Item	Reason
`archive/` (14.8k lines)	Git history + `v1-full` tag
`evaluation_gate.py` + `test_stage354_*` + `test_stage35_temporal_holdout_readiness` + `Dockerfile.evaluator` + `requirements-evaluator.lock`	Product mein koi use nahi
`gnn_model`, `hetero_gnn`, `temporal_gnn`, `phase5_hybrid_model`, `train_phase5.py`, Phase 5 artifacts	Apna benchmark haarta hai
`neo4j_service`, `neo4j_investigator`, `redis_service`, `seed_mesh.py`, `test_seed_mesh.py`	Default off, score nahi badalte
Grafana + Prometheus containers, dashboards	Decorative
`stream_service`, `/stream/transactions`, frontend feed page	Simulated feed
`mlops_pipeline`, `versioning`, `reproducibility`, `splits`, `missingness`	Sirf tests inhe use karte hain (training mein use ho to `utils.py` mein fold karo)
`multimodal_clip_faiss.py`	Wrapper, sirf test imports
Dependencies: `torch-geometric`, `neo4j`, `redis`, `sqlalchemy`, `psycopg2-binary`, `cryptography`, `sse-starlette`, `faiss-cpu`, `prometheus-client`	Upar ke removals ke saath
`.agents/`, `skills-lock.json`, `frontend/AGENTS.md`, `audit/`, `BUG_INVENTORY.json`	Tooling/paperwork; `.gitignore` mein daalo
`models/v2_1`, `models/stage34`, purani report JSONs	Live consumer nahi

Item	Reason
Frontend: `trust-graph` mock page, `settings`, `monitoring`, `SearchModal`, `ViewModeContext`	Fake ya low-value

CHANGE

Item	Change
Trust engine	Calibrated probability + cost-based thresholds + conformal. Blend, stacking, NNLR, duplicate weights, entropy "confidence", `model_disagreement` hatao
Serving	Sirf Phase 3 model. `model_version` /header/env switch hatao
CLIP	Serving mein cached embeddings; torch/transformers/Pillow `requirements-clip.txt` mein
Packaging	`pyproject.toml`, `pip install -e .`, saare `sys.path` hacks aur try/except imports hatao; trust engine FastAPI `Depends` se inject karo
Dossier stack	Phase 7 mein decide: delete (recommended) ya `investigations` page rakhkar rename (`policy_rules`, `dossier_generator`)
`advanced_ring_intelligence.py`	`research/` mein, agar results/docs mein referenced hai; warna delete
Scripts	Pehle data provenance check; phir `experiments/` mein 4–5 files

KEEP (non-toy story)

Temporal leakage tests, point-in-time features, 20-seed ablation + Holm correction, plain-counts control, GNN negative result (docs mein), isotonic calibration, conformal, TreeSHAP, FastAPI + auth, `check_no_hardcoded_metrics.py` + `verify_results.py`, aur 6 frontend pages (dashboard, transactions, returns, listings, fraud-rings, evaluation).

5. Execution plan

Rule: **ek phase = ek branch/PR**, aur har phase ke baad CI green. Chhote phases pehle, risky (trust engine, Phase 5) beech mein jab safety net ready ho.

Phase 0 — Safety net

- Tag `v1-full`, naya branch `lean`.
- Baseline: poora `pytest -m "not clip"`, `scripts/e2e_smoke_validation.py`, `npm run build`; pass counts note karo.
- `backend/fixtures/phase3_golden_baseline.json` ko raw aur calibrated probability regression check ke liye use karo (final *decision* Phase 4 mein jaan-boojh kar badlega).
- **Done when:** baseline numbers saved.

Phase 1 — Zero-risk cleanup

- `ci.yml` se archived test step hatao; `archive/` delete karo.
- `.agents/`, `skills-lock.json`, `frontend/AGENTS.md`, `audit/`, `BUG_INVENTORY.json`, `FRONTEND_IMPLEMENTATION_REPORT.md` hatao; `.gitignore` update.
- Unused deps: `sse-starlette`, `psycopg2-binary`; Dockerfile ka duplicate `faiss-cpu`; `pytest`/`httpx` → `requirements-dev.txt`.
- **Done when:** CI green, kuch bhi runtime behaviour nahi badla.

Phase 2 — Evaluation gate hatao

- Delete `evaluation_gate.py`, uske 2 test files, `Dockerfile.evaluator`, evaluator lock; `cryptography` dependency.
- Replace: `experiments/freeze_holdout.py` (~60–80 lines): date-based frozen split, predictions ka SHA-256, "test set ek baar chala" ka log.
- **Done when:** `grep -r evaluation_gate` khaali; CI green.

Phase 3 — Sirf Phase 3 model serve karo

- `scoring.py` ka phase5 branch, `features._build_hybrid_feature_row`, `model_loader` ka Phase 5/embeddings/hybrid-SHAP, `TRUSTSHIELD_SERVING_MODEL`, `model_version` field (schemas + frontend types) hatao.
- Delete 4 GNN files, `train_phase5.py`, 5 Phase 5 artifacts; `torch-geometric` hatao.

- 5 affected test files edit: `test_backend`, `test_security`, `test_services`, `test_leakage`, `test_phase2_suite`.
- GNN ka negative result `docs/experiments.md` mein, numbers `results.json` se.
- **Done when:** `/transaction/score` ka `raw_fraud_probability` golden baseline se match.

Phase 4 — Trust engine sahi karo (sabse important)

- Decision = calibrated probability vs thresholds. Review rule per order: flag karo jab `p ⩾ C_review / (C_review + order_amount)` (FN = order amount, FP = flat review cost); BLOCK threshold validation set par choose karo. Fixed numbers (jaise 0.75/0.40) hardcode mat karo.
- Conformal rakho (class-conditional); ambiguous set → REVIEW. Coverage ko "nominal" likho (out-of-time shift), hardcoded `conformal_coverage_guarantee=0.95` hatao.
- Hatao: `StackingRiskMetaLearner`, `NonNegativeLogisticRegression`, weights dict, backward-compat aliases, `calculate_confidence`, `model_disagreement`. Graph signals sirf **reason codes** ke roop mein (score mein nahi).
- Response schema chhota karo (abhi ~25 fields), frontend types update.
- Before/after table banao val set par: recall at 2%/5% review budget, decisions ka distribution.
- **Done when:** 0.95-prob fraud bina device sharing ke bhi HOLD/BLOCK mile; legit shared-address order sirf model ke hisaab se score ho (ALLOW, jab tak conformal set ambiguous na ho).

Phase 5 — Infra hatao

- Neo4j/Redis services + `neo4j_investigator` + `seed_mesh.py` delete; `/ready`, `/health` se unke fields hatao; `graph_topology` mislabel hatao.
- `docker-compose.yml` → sirf API (+ frontend agar chahiye); `docker/prometheus`, `docker/grafana` delete.
- `metrics_service` (19 call sites) → structured logging + ek simple timing decorator; `/metrics` aur uske 2 tests hatao; `prometheus-client` hatao. (Agar time kam ho, to is sub-step ko Phase 8 ke baad karo; baaki sab upar wale isse independent hain.)
- SSE: `stream_service`, endpoint, frontend feed page, 2 tests.
- **Done when:** `docker compose up` ek container; `/health` green.

Phase 6 — Serving decoupling aur packaging

- `pyproject.toml` banao, `trustshield_project` ko proper package, `pip install -e .` ; saare `sys.path` hacks, try/except import chains aur `_get_engine()` hatao; `pytest.ini` ka flat `pythonpath` hatao, tests mein package imports.
- Product catalog ek saved file (`models/product_catalog.parquet` ya CSV) se load karo, generator import hatao.
- `_resolve_graph_features` ek baar per request.
- **Done when:** `python -c "import backend.main"` bina sys.path hack ke chale.

Phase 7 — Orphan modules aur tests

- Delete: `mlops_pipeline` , `versioning` , `reproducibility` , `splits` , `missingness` , `multimodal_clip_faiss` ; `export_dataset` ko CSV-only rakho (sqlalchemy hatao); `advanced_ring_intelligence` → `research/` ya delete.
- `test_phase2_suite.py` ko chhota karo (sirf wo tests jo bache modules ko cover karte hain).
- Dossier stack (`investigation_service` 427 + agent 222 + rag 122 lines, aur hardcoded `PRESET_ENTITIES`): recommended delete, kyunki `/transaction/explain` ka SHAP narrative already investigator explanation deta hai. Agar `investigations` page rakhna hai to rename karo aur `PRESET_ENTITIES` hatao.
- **Done when:** import-graph script mein koi orphan non-test module nahi.

Phase 8 — Scripts aur models consolidation

- **Pehle provenance table banao:** `models/` aur `results.json` kis generator/dataset se bane. Jab tak ye clear na ho, koi generator delete mat karo.
- Agar train/serve mismatch mile (models purane generator par, PIT history v2.1 par): models ko ek hi dataset par retrain karo ya PIT store ko usi dataset par point karo.
- Phir ek pipeline rakho; 23 scripts → `experiments/` ke 4–5. Jo scripts `results.json` banati hain unka output regenerate karke diff karo; reproduce na ho to originals `experiments/legacy/` mein rakho.
- `verify_production_invariance.py` (deleted code ka absence check) delete; `models/` mein sirf serving artifacts, baaki gitignore ya release asset.
- **Done when:** `reproduce_all.py` fresh clone se chalti hai aur `results.json` regenerate karti hai.

Phase 9 — Frontend

- 6 pages rakho: dashboard, transactions, returns, listings, fraud-rings, evaluation. `trust-graph` ko `/fraud-rings` ke real data se chalao ya delete. `settings`, `monitoring`, `SearchModal`, `ViewModeContext` hatao; `investigations` page Phase 7 ke dossier decision ke saath jaata hai. `demoAdapter.ts` chhota rakho agar bina backend ke deploy karna ho.
- **Done when:** `npm run build` + `lint` pass; koi page hardcoded fake data nahi dikhata.

Phase 10 — Docs

- README ko nayi reality se match karo (Neo4j/Redis/Grafana/GNN-in-serving ke mentions hatao, `BEGIN GENERATED` markers saaf).
- `docs/ENGINEERING_NOTES.md`: leakage audit, GNN negative result, decisions. `MODEL_CARD.md` rakho.
- **Done when:** README ka har claim code ya `results.json` se trace ho.

6. Target end state

```
backend/        main.py, routers/, features.py, auth.py, schemas.py,
model_loader.py
trustshield/    (package) generators, graph_features, baseline_model,
calibration,
                trust_engine (slim), shap_explainer, point_in_time_history,
multimodal_scoring
experiments/    4–5 scripts + research/
models/         serving artifacts only
frontend/       5 pages
docs/           README, MODEL_CARD, ENGINEERING_NOTES
```

Dependencies:

- `requirements.txt` (serving): numpy, pandas, scikit-learn, xgboost, networkx, shap, joblib, fastapi, uvicorn, pydantic. (`scipy` / `pyarrow`: sirf agar serving mein use ho — verify.)
- `requirements-dev.txt`: pytest, httpx, flake8.
- `requirements-clip.txt` (offline embeddings): torch, transformers, Pillow.

7. Risks aur rollback

Risk	Mitigation
Tests tootenge (phase5, metrics, stream, neo4j refs)	Har phase mein unhi tests ko edit karo; green hone par hi merge
Trust engine change se decisions badlenge	Intentional; before/after table Phase 4 mein; golden baseline sirf probabilities par
Script ya module jo maine docstring se orphan maana, actually kaam ka nikle	Delete se pehle `grep -r` + `pytest`; tag `v1-full` se wapas milega
`models/` cleanup ke baad Docker `COPY models/` mein kuch missing	Phase 8 ke baad container build + smoke test
Frontend types backend response se out of sync	Phase 4 aur 9 mein `npm run build` + smoke

8. Resume story (cleanup ke baad)

"Temporally-safe fraud scorer: 20-seed ablation se dikhaya ki graph features ka lift modest (+0.011) hai aur plain counts jitna hi; GNN ko test karke reject kiya; calibrated probabilities + conformal + SHAP ke saath ek FastAPI service." Ye honest, measurable aur toy-jaisa nahi hai. Infra ceremony ko hatane se ye story zyada saaf dikhti hai.

Resume mein sirf measured numbers: dataset 5,000 buyers / 50,000 orders (README ke hisaab se, 100k+ nahi), conformal ko "nominal coverage" likho, aur "sub-500MB" ya latency tabhi likho jab `docker images` aur benchmark se naapa ho. "Mathematically prevents leakage" ki jagah "leakage tests se verified" likho.

9. Is review ki limits

- Maine `main.py`, `features.py`, `routers/scoring.py`, trust engine, `model_loader.py`, README, CI aur `multimodal_scoring` ke relevant hisse line-by-line padhe. Baaki files (scripts, frontend pages, baaki tests) ke liye structure, docstrings, import graph aur targeted greps use kiye. Poori ~47k lines line-by-line nahi padhi.
- Sandbox mein pytest/fastapi nahi tha, isliye tests run nahi kar paya. `data/` zip mein nahi tha.

- Dependency/image size ke numbers estimates hain, measured nahi.

10. Cross-review: ek aur AI report ke galat claims

Dusri report ke ye claims code se match nahi hue; upar ke plan mein inhe theek kiya gaya hai.

Claim	Verdict	Evidence
`.env.example` mein `NEO4J_ENABLED` / `REDIS_ENABLED` false hain	**Galat**	Dono `true` hain; sirf code default false hai
`seed_graph.cypher` / `seed_redis.txt` zip mein the	**Galat**	Zip mein hain hi nahi (`ls` se verified); gitignored. Local disk par hon to wahan delete karo
Runtime deps 17 → 10	**Galat**	`requirements.txt` mein 26 packages hain; serving ke baad ~12
Purane generators delete, v2.1 "modern"	**Risky**	`train_and_save_models.py` purane generators use karta hai; pehle provenance check (Phase 8)
BLOCK ≥ 0.75, HOLD ≥ 0.40, "model disagreement > 0.45"	**Invented**	Thresholds cost-based hone chahiye; detectors hatne ke baad disagreement ka matlab nahi
Resume: "100k+ transactions", "sub-500MB", "under 25 ms", "pre-registered", "mathematically preventing leakage"	**Unverified / overclaim**	README: 50,000 orders; baaki measured nahi
5 pages, lekin dossier aur returns endpoints bina UI ke	**Inconsistent**	Ab 6 pages + dossier decision (Phase 7)
Phase 0 mein golden baseline regenerate karo	**Mat karo**	Fixture current output se overwrite ho jaata hai; committed fixture + `test_production_baseline_regression.py` use karo

Unki report se jo achha tha wo adopt kiya: trust engine ko FastAPI `Depends` se inject karna, `docker build` ke baad image size naapna (Phase 10), aur dependency ko serving / dev / clip mein todna.